6.7920: Reinforcement Learning **Lecture Date: October 8, 2026**

L09. Model-Free Policy Learning

Lecturer: Cathy Wu *Scribe: Jinhee Won*

1 Introduction

Previously, we studied how to estimate the value of a fixed policy using Monte Carlo and temporal-difference methods. We now turn to *control*: learning a policy that maximizes expected return.

Recall the distinction:

- **Policy iteration:** alternate between evaluating a policy and improving it.
- **Value iteration:** repeatedly apply the optimal Bellman operator, without fully evaluating each intermediate policy.

Both methods traditionally require a model. For example, extracting a greedy policy from a state-value function requires

$$\pi(s) \in \arg\max_a \left\{ r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V(s')] \right\}.$$

Without knowing the rewards and transition probabilities, how can we compare actions?

2 State-Action Value Functions (Q-function)

We introduce a new term called the Q-function, which is very similar to the value function be we instead also take in the action as a parameter.

2.1 Definition and Intuition

The state-action value function is

$$Q^\pi(s,a) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \,\middle|\, S_0 = s,\ A_0 = a,\ \text{follow } \pi \text{ thereafter} \right].$$

Unlike $V^\pi(s)$, which follows π immediately, $Q^\pi(s,a)$ specifies the first action and follows π afterward.

For a deterministic policy,

$$V^\pi(s) = Q^\pi(s, \pi(s)).$$

Also,

$$Q^\pi(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V^\pi(s')].$$

The optimal Q-function satisfies

$$Q^*(s,a) = \max_\pi Q^\pi(s,a), \qquad V^*(s) = \max_a Q^*(s,a).$$

An optimal policy can therefore be extracted directly:

$$\pi^*(s) \in \arg\max_a Q^*(s,a).$$

2.2 Bellman Operators

This leads to the Bellman Operators for the Q-function. For a deterministic policy π, define

$$(\mathcal{T}^\pi Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[Q(s', \pi(s'))].$$

For a stochastic policy, also average over $a' \sim \pi(\cdot \mid s')$.

The optimal Bellman operator is

$$(\mathcal{T}Q)(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[\max_{a'} Q(s',a')\right].$$

As with state-value functions, these operators are contractions in the sup norm, with fixed points

$$Q^\pi = \mathcal{T}^\pi Q^\pi, \qquad Q^* = \mathcal{T} Q^*.$$

2.3 Benefits and Tradeoffs

A tabular state-value function stores $|S|$ entries, whereas a Q-function stores $|S||A|$ entries, making the memory storage of a Q-function higher.

However, this additional memory makes action selection convenient: given an estimated Q-function, we can select a greedy action without consulting the model.

3 Dynamic Programming with Q-Functions

We can translate the algorithms to solve dynamic programming from the value function to the Q function.

3.1 Policy Iteration with Q-Functions

Starting from a policy π_0, alternate:

1. **Policy evaluation:** compute Q^{π_k}, for example by repeatedly applying $\mathcal{T}^{\pi_k}$.
2. **Policy improvement:** set
$$\pi_{k+1}(s) \in \arg\max_a Q^{\pi_k}(s,a).$$

With exact evaluation, the usual policy-improvement guarantees continue to hold.

3.2 Q-Value Iteration

Instead of fully evaluating each policy, repeatedly apply the optimal Bellman operator:

$$Q_{k+1}(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[\max_{a'} Q_k(s', a') \right].$$

4 Q-Learning

However, the goal of this lecture was to go model-free! The Q-function lets us extract a greedy policy without a model, but the Q-value iteration update still requires an expectation over the next state. To remove this dependence on the model, we replace the expectation with samples from interaction.

4.1 From Q-Value Iteration to Q-Learning

Suppose we observe a transition $(S_t, A_t, R_{t+1}, S_{t+1})$. We construct the sampled Bellman target

$$Y_t = R_{t+1} + \gamma \max_{a'} Q_t(S_{t+1}, a').$$

Rather than overwrite our estimate with this noisy target, we take a small step toward it:

$$Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) + \alpha_t(S_t, A_t)\delta_t,$$

where

$$\delta_t = \underbrace{R_{t+1} + \gamma \max_{a'} Q_t(S_{t+1}, a')}_{\text{sampled Bellman target}} - \underbrace{Q_t(S_t, A_t)}_{\text{current estimate}}$$

is the temporal-difference (TD) error.

All other entries remain unchanged. If S_{t+1} is terminal, the bootstrap term is zero.

This gives the basic Q-learning procedure: choose an action, observe a transition, and update the corresponding Q-value. However, we have not yet specified how to choose actions.

4.2 Exploration vs. Exploitation

A natural choice is to always take an action with the highest estimated Q-value. But these estimates may initially be inaccurate: if we never try an action, we may never discover that it is better!

This creates a tradeoff between:

- **Exploitation:** selecting actions with high estimated value to obtain reward.
- **Exploration:** trying actions to learn more about their values.

A purely greedy agent exploits its current estimates, but may fail to collect the data needed to improve them.

4.3 ϵ-Greedy Q-Learning

One simple way to balance exploration and exploitation is an ϵ-greedy policy.

Initialize Q_0 and observe an initial state S_0. At each step t:

1. With probability $1 - \epsilon$, choose
$$A_t \in \arg\max_a Q_t(S_t, a).$$
With probability ϵ, sample A_t uniformly from the available actions.
2. Take action A_t and observe reward R_{t+1} and next state S_{t+1}.
3. Apply the Q-learning update:
$$Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) + \alpha_t(S_t, A_t)\left[R_{t+1} + \gamma \max_{a'} Q_t(S_{t+1}, a') - Q_t(S_t, A_t)\right].$$
Set the bootstrap term to zero if S_{t+1} is terminal.
4. Continue from S_{t+1}, or reset the environment if the episode has ended.

The learned policy is obtained by acting greedily with respect to the final Q-function.

4.4 Convergence Conditions

Do these sampled, incremental updates still converge to Q^*? In a finite discounted MDP with bounded rewards, tabular Q-learning converges to Q^* almost surely if:

- Every state-action pair is visited infinitely often.
- For every pair (s, a), the learning rates over its successive visits satisfy the Robbins–Monro conditions:
$$\sum_{n=1}^{\infty} \alpha_n(s, a) = \infty, \qquad \sum_{n=1}^{\infty} \alpha_n(s, a)^2 < \infty.$$

The first condition ensures sufficient coverage, and is ensured using the epsilon-greedy policy. The learning-rate conditions ensure that updates remain large enough to correct initial errors, while shrinking enough to control sampling noise. We can see this analysis using stochastic approximation, which we will cover next lecture.

5 SARSA and On-Policy vs. Off-Policy Learning

The ϵ-greedy policy is just one way to select actions; we could also use another exploration strategy, such as softmax. However, there is a separate choice hidden in our algorithm: should the update use the *best estimated next action*, or the *next action actually selected*?

This distinction leads to Q-learning and SARSA, and more generally, off-policy and on-policy learning.

5.1 Definitions

The **behavior policy** generates the actions used to collect data. The **target policy** is the policy whose value is being learned.

- **On-policy:** the behavior and target policies agree.
- **Off-policy:** they can differ.

5.2 Why Q-Learning Is Off-Policy

Q-learning may collect data using an ϵ-greedy policy, but its target uses the greedy next action:

$$R_{t+1} + \gamma \max_{a'} Q_t(S_{t+1}, a').$$

It therefore learns about greedy behavior while potentially acting exploratorily.

5.3 SARSA: Bootstrap from the Actual Next Action

SARSA samples A_{t+1} from its behavior policy and uses

$$\delta_t^{\text{SARSA}} = R_{t+1} + \gamma Q_t(S_{t+1}, A_{t+1}) - Q_t(S_t, A_t).$$

Because the next action is sampled from the same policy used for interaction, standard SARSA is on-policy.